

PROMPT FOR STAGE 4 OF ALGORITHM → CREATING THE “SMART GRID FILTER”:

You are a traffic-camera calibration expert and assistant.

Your main objective is to generate a filter with speed, direction, and probability-presence thresholds using the velocity matrix data attached below. Return JSON only, no commentary.

CAMERA: [camera_id]
FRAME: [480] x [270] px
FPS: [15]

Inputs:

Filter 1: You have also been given a preliminary filter, which illustrates primary directional traffic flow.

Velocity Matrix: CSV's attached, including real-world units that are already computed. These include x_px, y_px, speed_mph, direction_deg [matrix_[camera_id].csv]. (Last two variables were calculated with pixel-to-real-world approach)

Reference Image (attached): frame representative of the entire clip with trajectories overlaid. These trajectories are color-coded by direction. This image is for visual context ONLY and for judging road edges when the matrix data is sparse.

Your Task:

Begin by dividing the “Filter 1” region into a 30x30 px grid. For each cell:

- 1) Pull all matrix rows whose (x_px, y_px) fall in this cell.
 - n_observations = row count. If n_observations = 0, SKIP DIRECTLY to step 5.
- 2) raw_speed_mph = median(speed_mph), raw_direction_deg = median(direction_deg) of those rows.
- 3) probability_presence (0.0-1.0): based on n_observations relative to neighboring cells and the reference image. This is what separates the carriageway from the shoulder. Do not smooth this across neighbors
- 4) Smoothing: apply 2D Gaussian smoothing across the grid instead of a flat blend. For each cell, compute a weighted average of raw_speed_mph (and separately raw_direction_deg) over all cells within a 2-cell radius (5x5 window) in the same Filter 1 region that have n_observations > 0. A neighbor at row/column offset (Δr , Δc) gets weight $\exp(-(\Delta r^2 + \Delta c^2) / (2\sigma^2))$ with $\sigma = 1.0$,

normalized so all weights in the window sum to 1. This cell's own value sits at offset (0,0), so it naturally dominates its own estimate while nearby cells pull it gently toward theirs — no hard jump between neighbors. Report the result as a threshold range: ± 4 mph, $\pm 12^\circ$ around the Gaussian-smoothed center. Never include cells from outside this cell's Filter 1 region in the window

Adjacent cells' ranges should overlap, never hard-jump. Never smooth across a Filter 1 region boundary

5. Cells with `n_observations = 0`:

- if visually on-road, derive speed/direction range entirely from neighbor smoothing, mark "interpolated": true. If visually off-road, set `probability_presence < 0.05`, leave speed/direction null, mark "interpolated": false.

OUTPUT:

```
{
  "camera_id": "string",
  "grid_cell_size_px": 30,
  "regions": [
    {
      "filter1_region_id": "string",
      "direction_label": "string",
      "cells": [
        {
          "cell_id": "r{row}_c{col}",
          "pixel_bounds": {
            "x_min": int,
            "x_max": int,
            "y_min": int,
            "y_max": int
          },
          "speed_range_mph": [number, number] | null,
          "direction_range_deg": [number, number] | null,
          "probability_presence": number,
          "n_observations": integer,
          "interpolated": boolean
        }
      ]
    }
  ]
}
```

